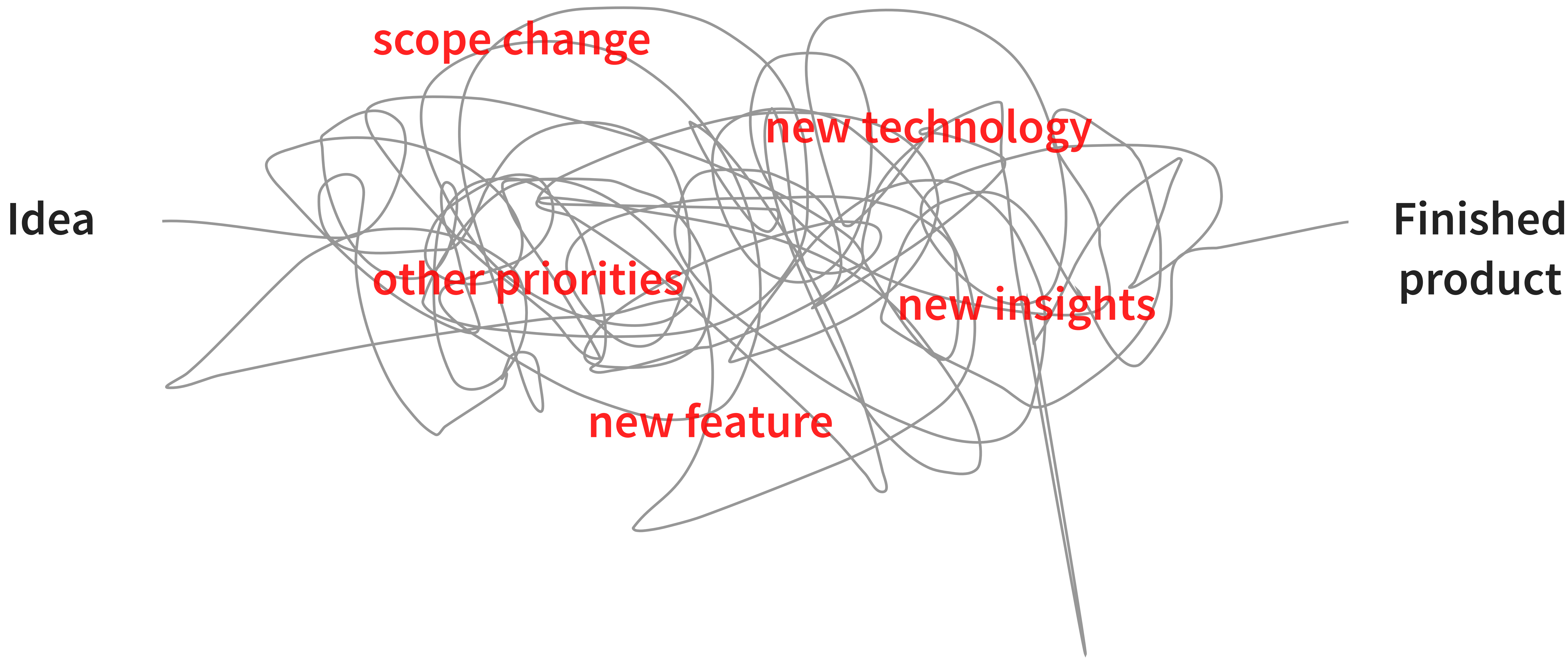
Introduction to Design systems

Process

In our current process of creating a new product we are facing a lot of obstacles. Like a domino effect, every little change affects the next step. This can happen due to many factors and we need to come up with a way to avoid losing time in process.



Goals

Like the development team the UX/UI design team also has it's constraints. The compromise between the two will determine the end product. This should be a clear and easily modifiable path along the way.

Developers focus:

Logic

Security issues

Performance

Designer focus:

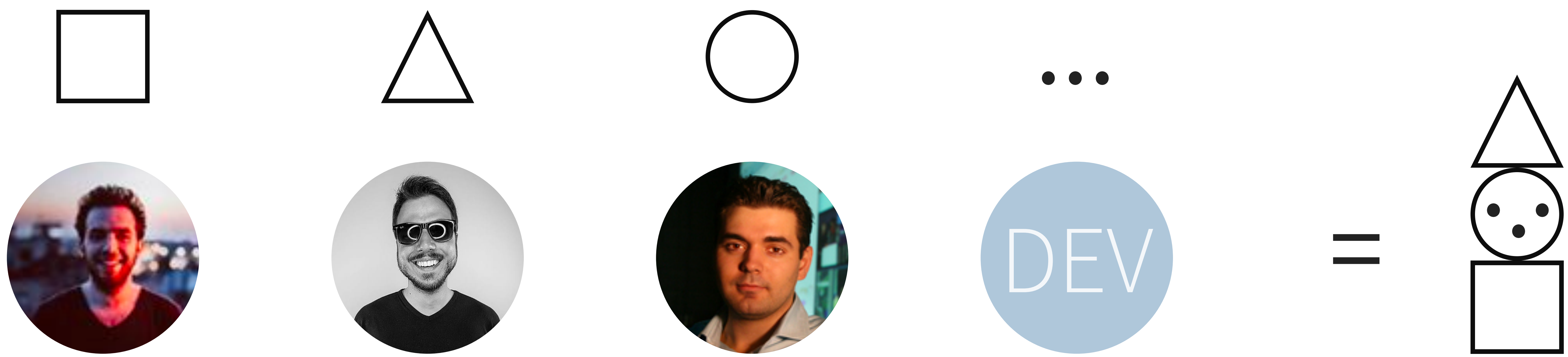
Usability

User experience

User Interface

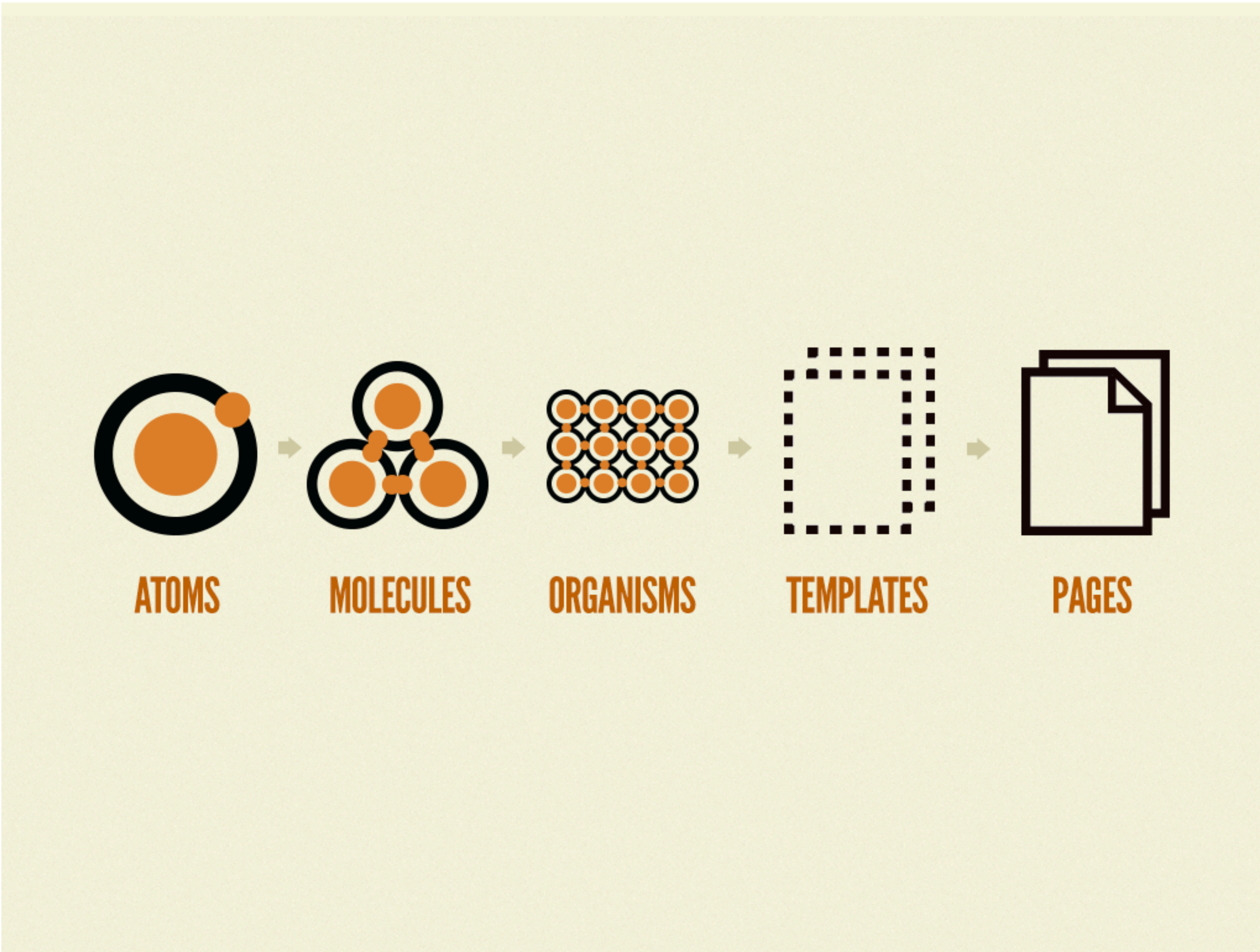
Inconsistency

In the end product we are battling with one of our biggest pain points, which is inconsistency. That is a foundation for bad UX and a lot of other issues we are facing. To validate if my assumption about this is true I had to get the whole picture. That is why I started making interface inventory of the most used components found on our dashboard, and placing them in a design library.



Atomic Design methodology

I started by following Brad Frost's atomic design methodology and gathered what he describes as atoms - single elements like fields, icons, switches, text styles, etc. When atoms are combined they can create molecules like modals, tables, cards and so on, which can further evolve to organisms, templates and eventually to whole pages.

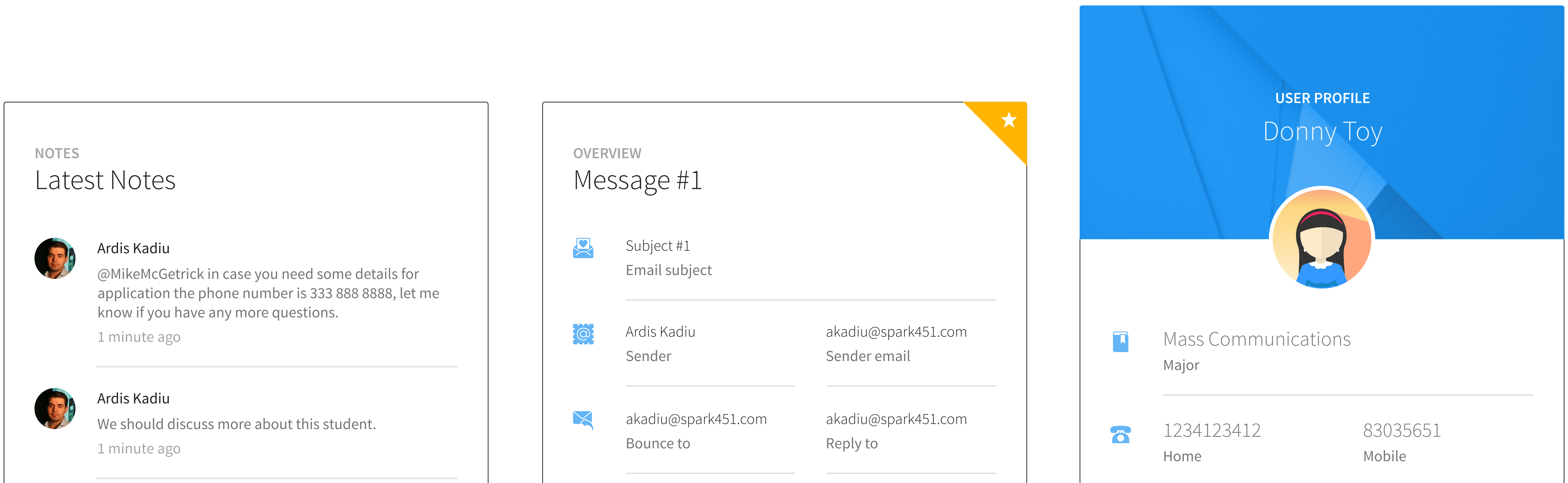


Not cool.

As my design library was growing bigger I noticed my assumption was right and that the inconsistency is present in many places.
You don’t need to be a designer to realize having 20 different font styles is a bad idea.
All of those styles represent time, money and energy.

Aa Aa Aa Aa

Aa **Aa** **Aa** **Aa**



Much better

By getting rid of the custom elements such as colors, font sizes and weights I came to a clean and usable design library capable of building our most used components and pages. Using this design library will help us work much faster while delivering consistency at the highest level.

LUM-DISPLAY-2

42pt Light FG-3 (0.54)

LUM-DISPLAY-1

34pt Light FG-2 (0.87)

LUM-HEADLINE

22pt Light $mat-blue: 500: #2196f3

22pt Light FG-2 (0.87)

22pt Light FG-3 (0.54)

LUM-TITLE

18pt Light FG-2 (0.87)

LUM-BODY-1

16pt Regular $mat-blue: 500: #2196f3

16pt Regular FG-2 (0.87)

16pt Regular FG-3 (0.54)

16pt Regular FG-5 (0.26)

BUTTON.LU-SML

14pt Semibold $mat-blue: 500: #2196f3

14pt Semibold FG-3 (0.54)

14pt Semibold FG-4 (0.35)

14pt Semibold FG-2 (0.87)

LUM-BODY-2

14pt Regular $mat-blue: 500: #2196f3

14pt Regular FG-2 (0.87)

LUM-DISPLAY-3

60pt Semibold

LUM-DISPLAY-2

42pt Light

LUM-DISPLAY-1

34pt Light

LUM-HEADLINE

22pt Light

LUM-TITLE

18pt Light

LUM-BODY-1

16pt Regular

BUTTON.LU-SML

14pt Semibold

LUM-BODY-2

14pt Regular

LUM-SUBHEAD

12pt Semibold

LUM-CAPTION

12pt Regular

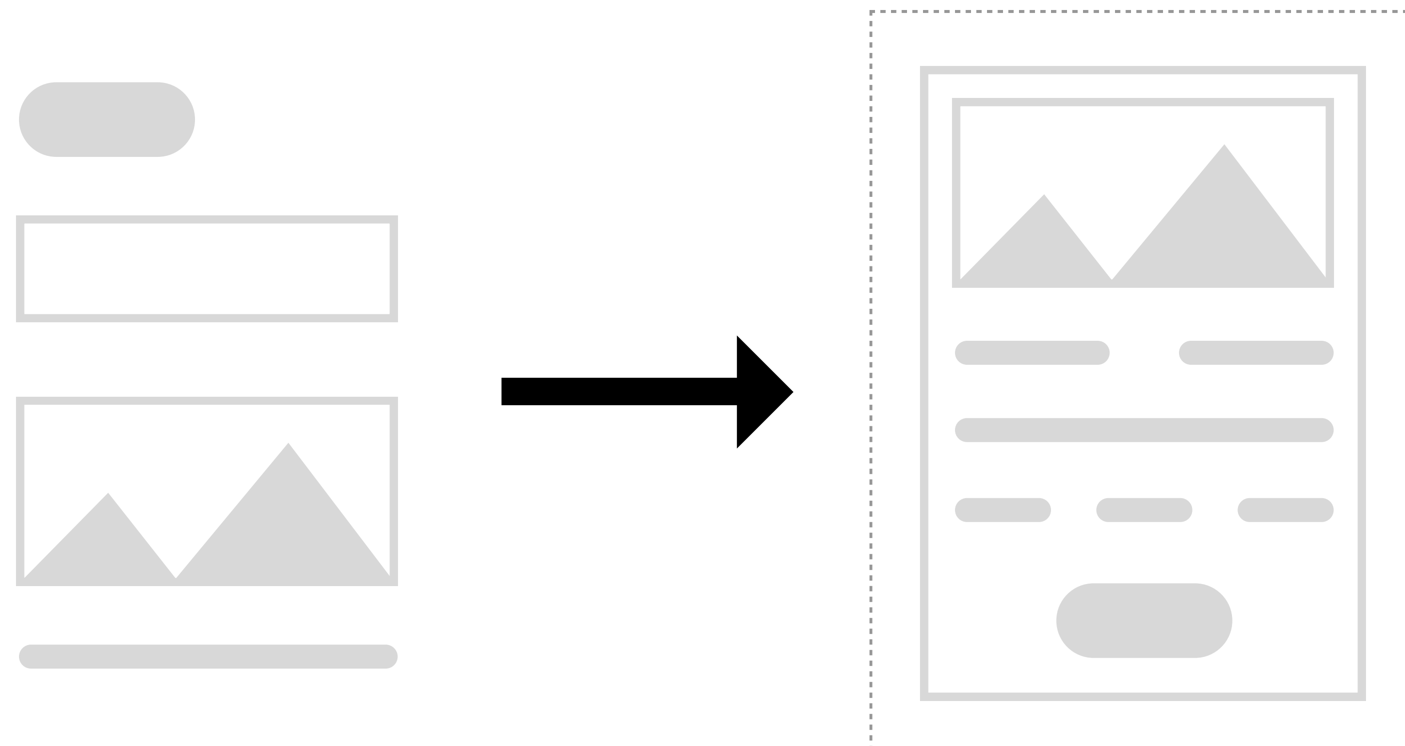
MAT-TOOLTIP

10pt Regular

So, what is a design system?

If we apply the same principles you saw in the design library to the engineering part, with clear guidelines that are scalable, we will have a design system in place. And by establishing a design system we will be able to work more efficiently, effectively and create a more cohesive experience as a result. Most importantly this will lead to **faster product launches and less QA fixes**, which gives us **more room to focus on the business logic**.

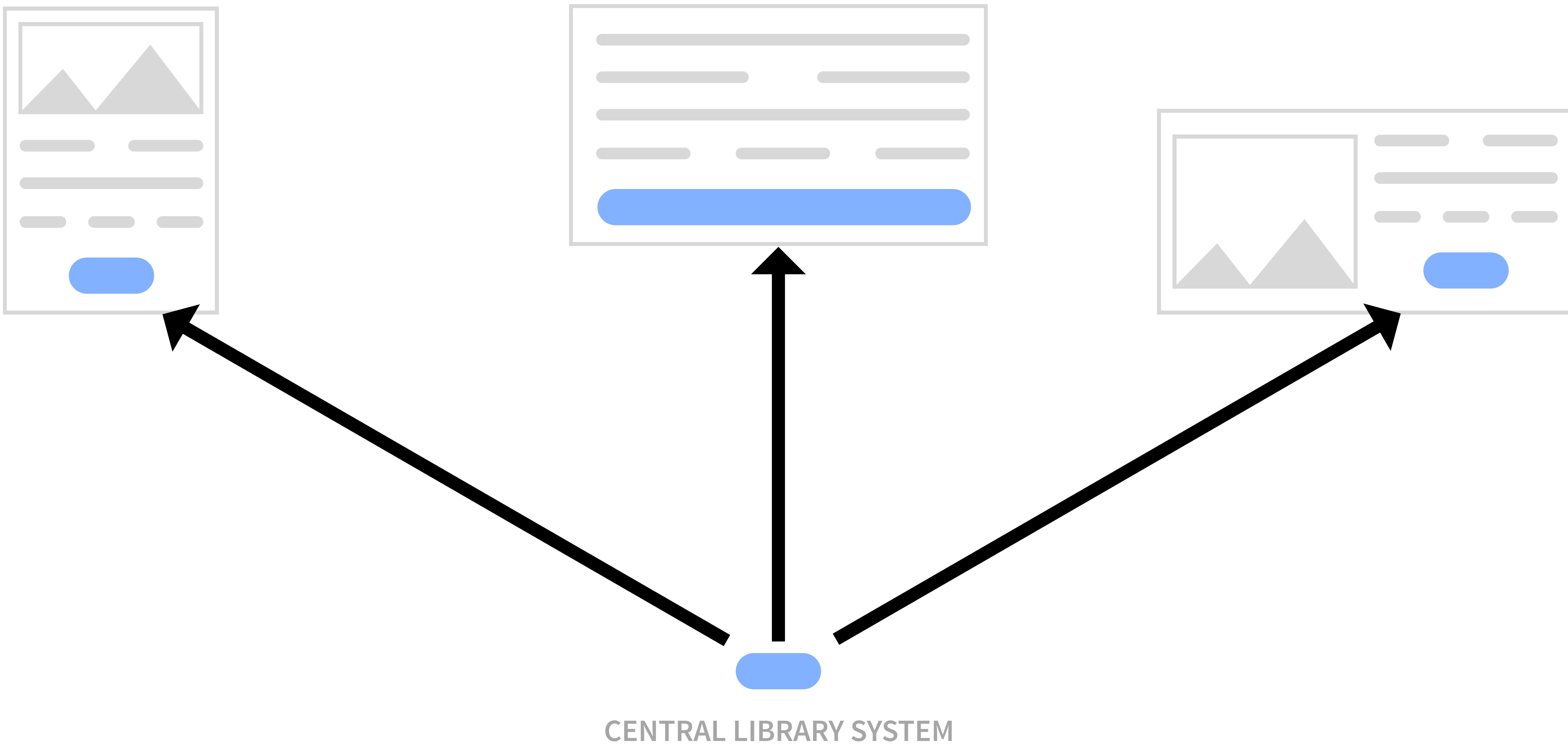
A design system is a series of components that can be reused in different combinations. It allows you to manage design at scale.



Cohesive experience

All of the master elements are kept in one, centralized place. And by updating them there, they also automatically get updated everywhere they are being used - in the designs, live sites and other places.

We are aiming to be completely WCAG 2.0 AA compliant. All of the testings to achieve those standards can be done in one place. Having a design system would make updating the changes a breeze.

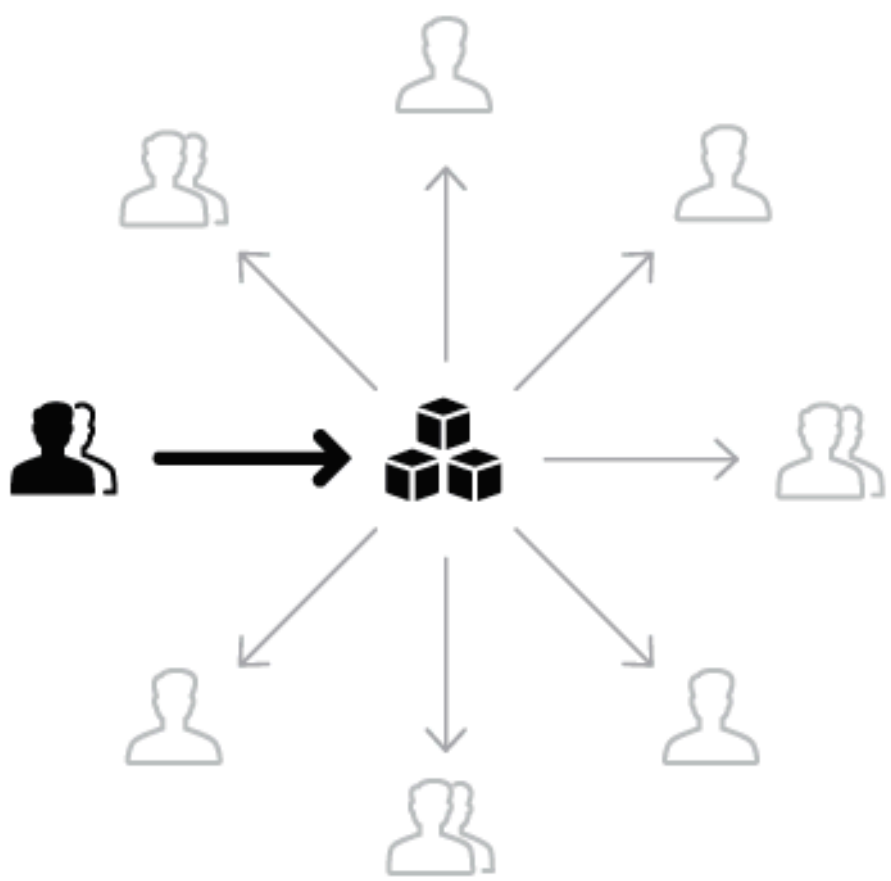


Team models for design systems

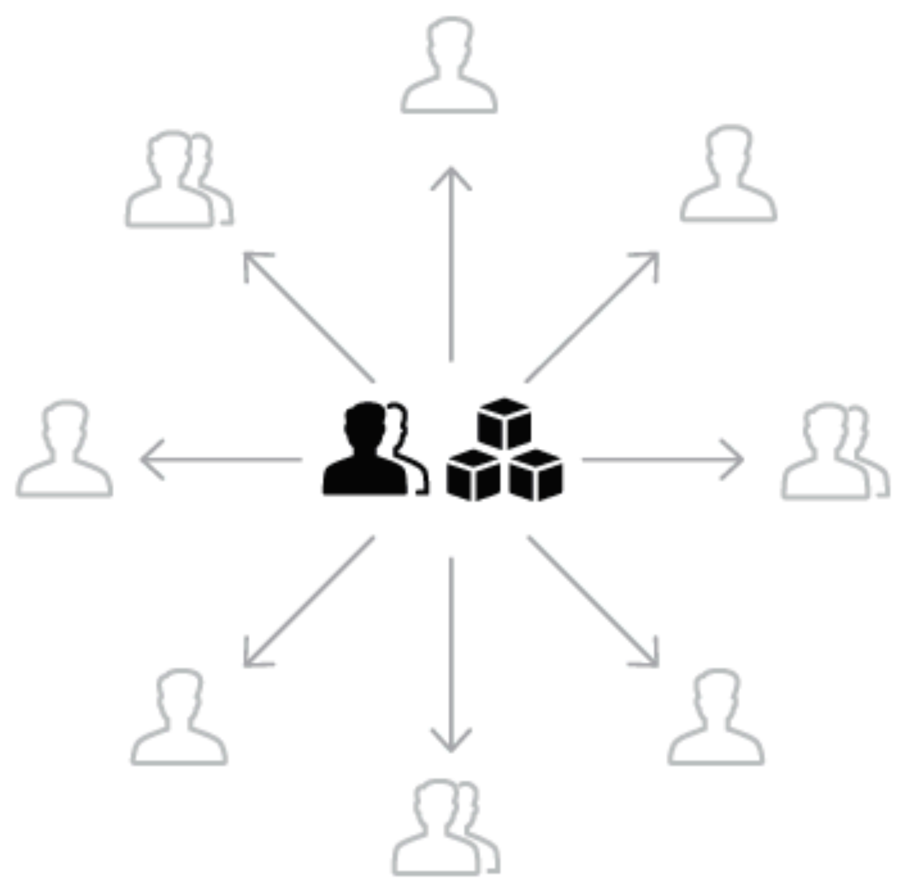
There are different models for design system. An **Overlord #1** - at this moment would be me, since I am the only person working on it and maintaining it, which may not be ideal for a big company. **Centralized #2 -** where a dedicated team would work on the design system. And **Federated #3**, where only certain people will be able to manage the system when needed.

A design system is only as good as its documentation, so it's really important we create a place that is easy to use and keep updated. Think of a design system as a bridge between designers and developers. By using a shared language we will understand the problems better, move things faster and perform more efficiently.

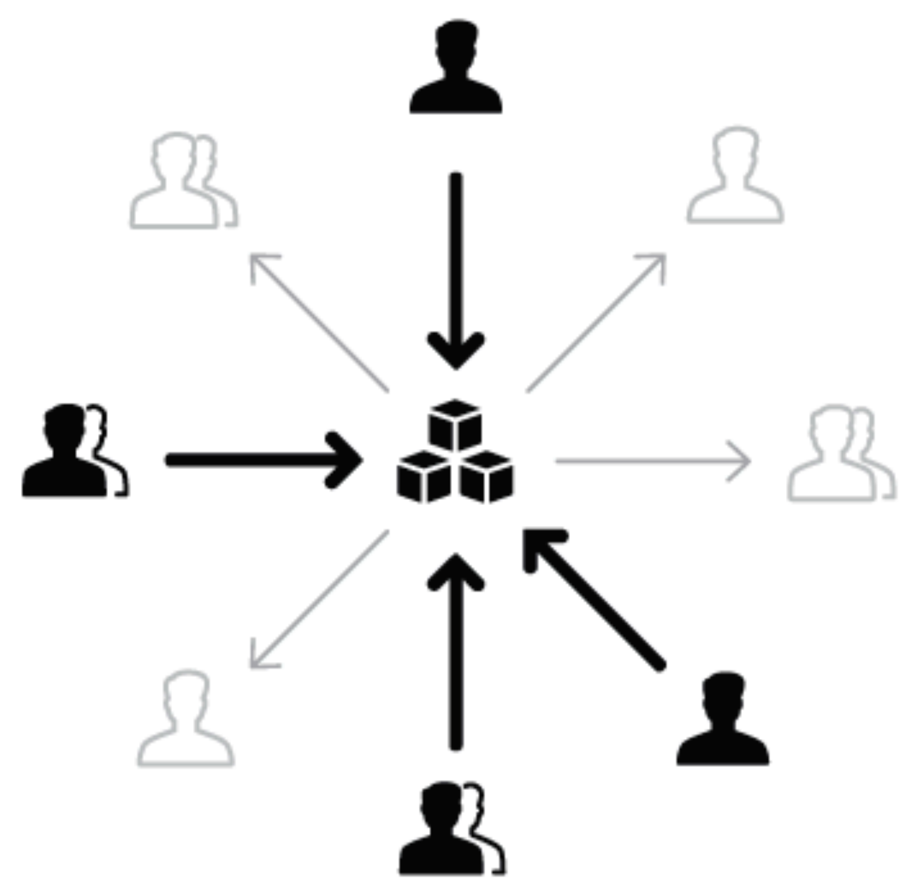
Important thing is that the guidelines and terminology about the system are adopted and followed by the whole team. Otherwise a design system can't survive. We should clearly define what is the intention behind each component and how should it be used. Also, since we use multiple technologies, we need to make our design system agnostic so it's scalable.



TEAM MODEL #1
SOLITARY (OVERLORD)



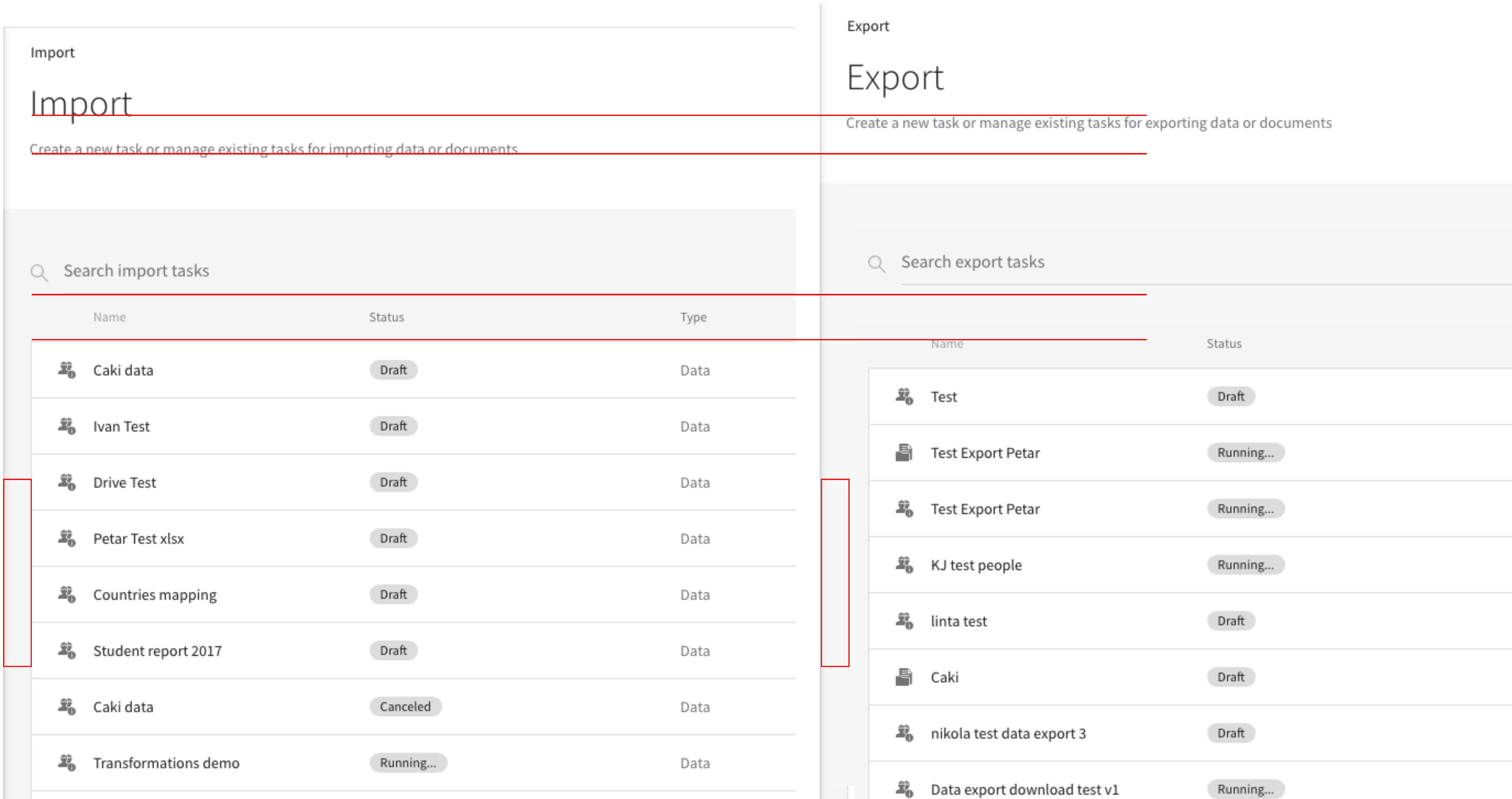
TEAM MODEL #2
CENTRALIZED



TEAM MODEL #3
FEDERATED

Components inconsistency

We have many developers building many products. One dev would build out a feature and another dev would build another feature, and those features end up on the same page, and you can tell they are built by 2 different people. Maybe it's the difference in size of a button, or a color but the inconsistency is very present. This is due to leaving to much room for interpretation. To tackle this issue we need to think bigger than pixels, rems or percentages.

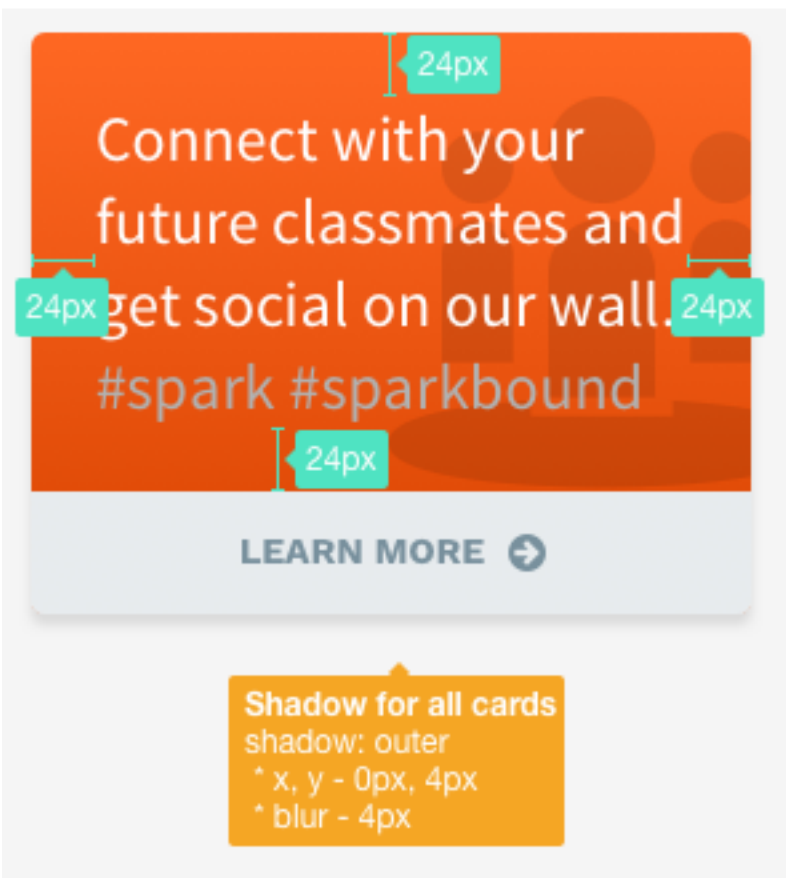


Designers ----> Developers

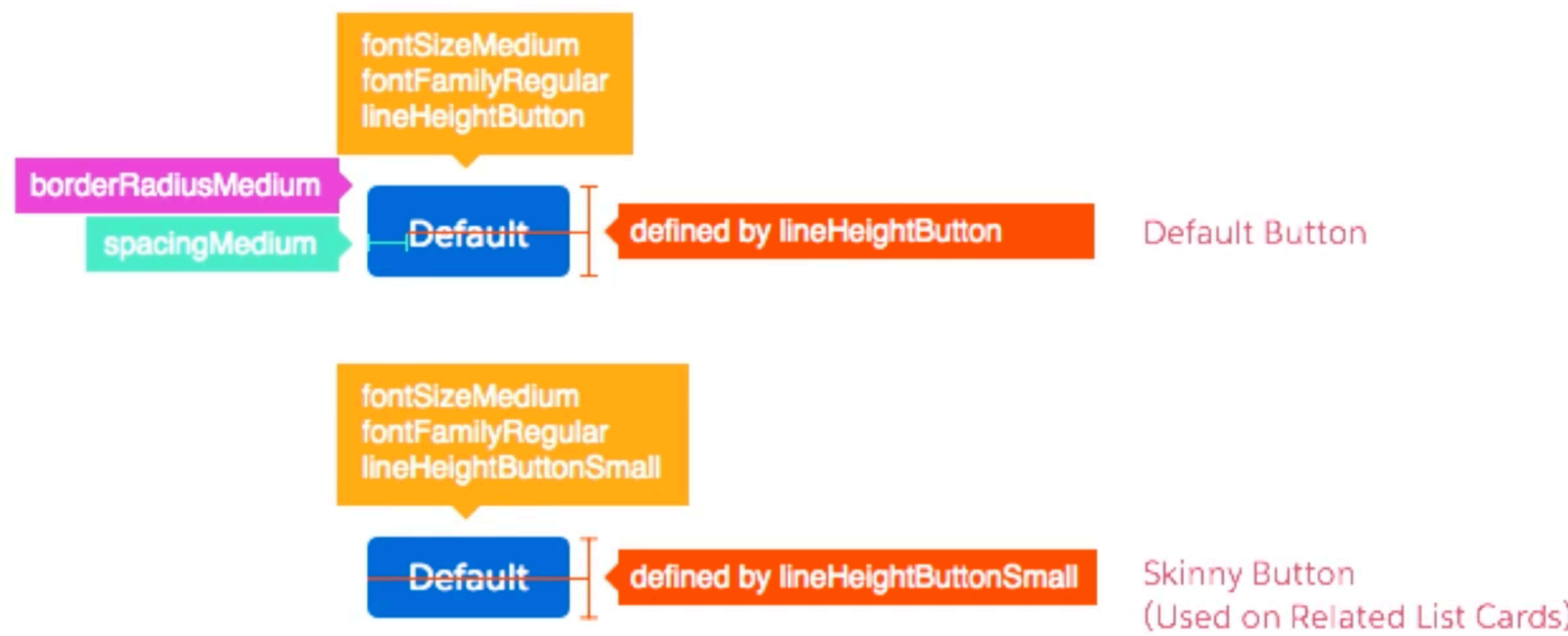
Reducing the system components to simpler forms makes them more reusable and replicable. Here we can abstract things. Pieces of code that are reusable on different sections of the dashboard can be stored as variables. For example spacings, sizing, rounded corners, font weights… basically all of the things that go behind a semicolon in css can be stored as variable.

So when creating a design specification rather than saying that spacing on a button is 16px instead we would say spacing medium, which is the name of the token used to define the spacing.

Old way



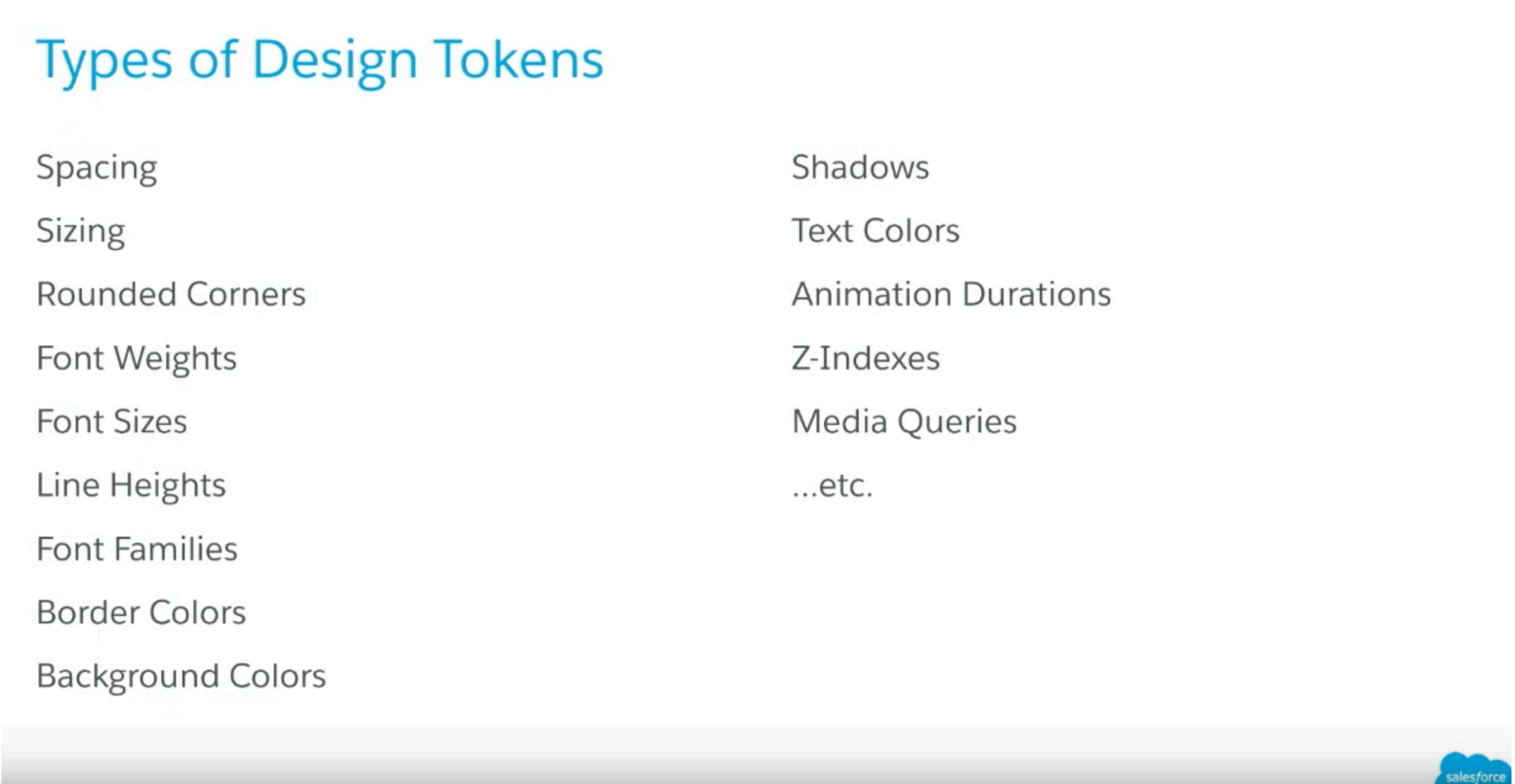
The new way



Types of tokens

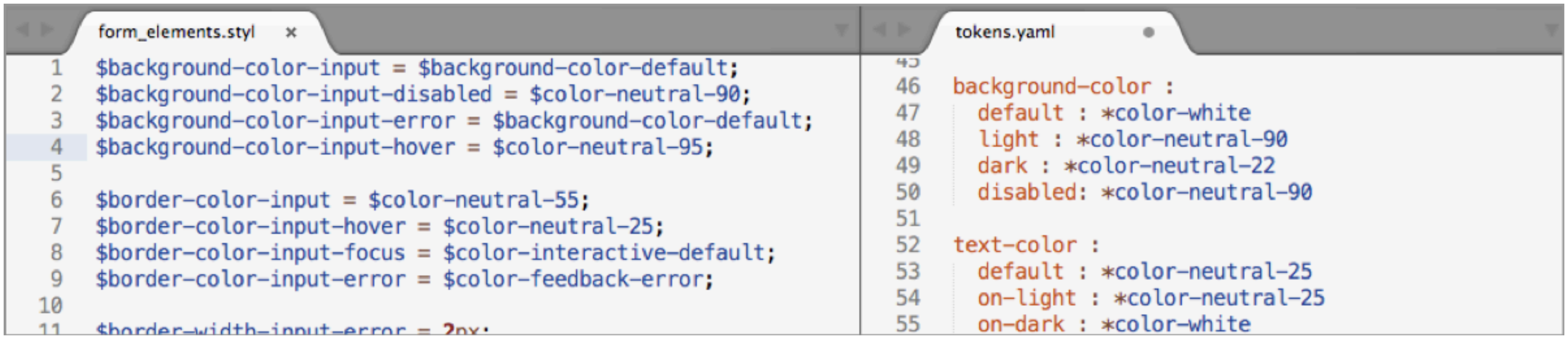
These are called design tokens in lightning design system made by salesforce. No more hard coded values which leads to eliminating errors and improves communication between designers and developers. All of the development teams will get updates in their system automatically this way and they don't have to touch a single thing. Design tokens enable scaling of the design across all the permutations.

These are some of the token types that are used by salesforce. Not all things should be tokens. What qualifies some properties to become one is the number of times they're being used on our dashboard. In average anything that appears on more than 4 places is a good candidate for a token.



What is the relationship between a class and a token?

The css can consume tokens, so for a component like a button the class would consume a background token, a spacing token, foreground color and radius. This is a good example of abstract thinking and it’s benefits.



form_elements.styl

```
$background-color-input = $background-color-default;
$background-color-input-disabled = $color-neutral-90;
$background-color-input-error = $background-color-default;
$background-color-input-hover = $color-neutral-95;

$border-color-input = $color-neutral-55;
$border-color-input-hover = $color-neutral-25;
$border-color-input-focus = $color-interactive-default;
$border-color-input-error = $color-feedback-error;

$border-width-input-error = 2px;
```

tokens.yaml

```
background-color :
  default : *color-white
  light : *color-neutral-90
  dark : *color-neutral-22
  disabled: *color-neutral-90

text-color :
  default : *color-neutral-25
  on-light : *color-neutral-25
  on-dark : *color-white
```

Same properties, different tokens.

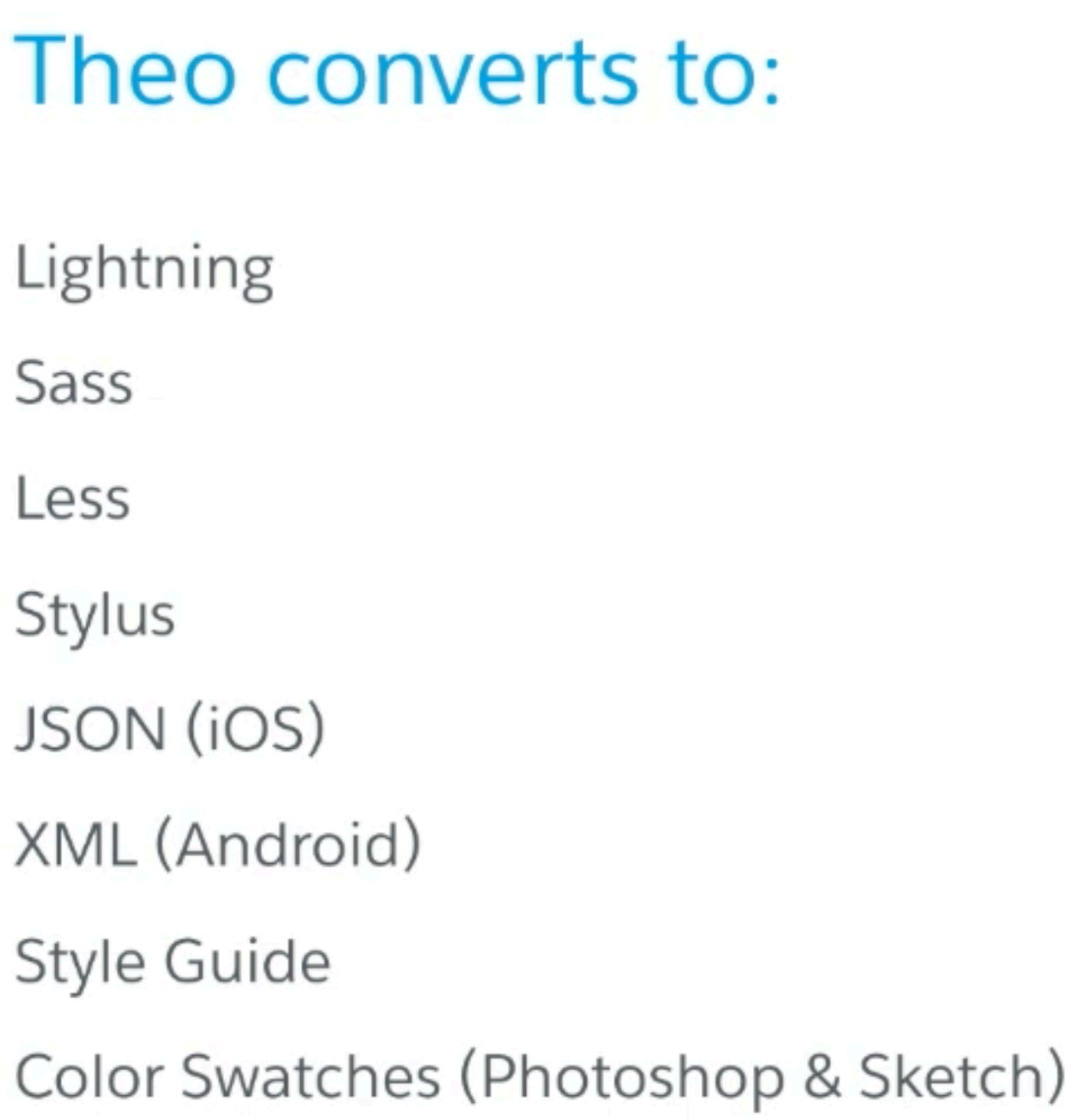
As scaling occurs we might need several tokens that have the same properties, but with different names. This will help us avoid changes we made to one component to affect another that is using same properties, but for different purposes.

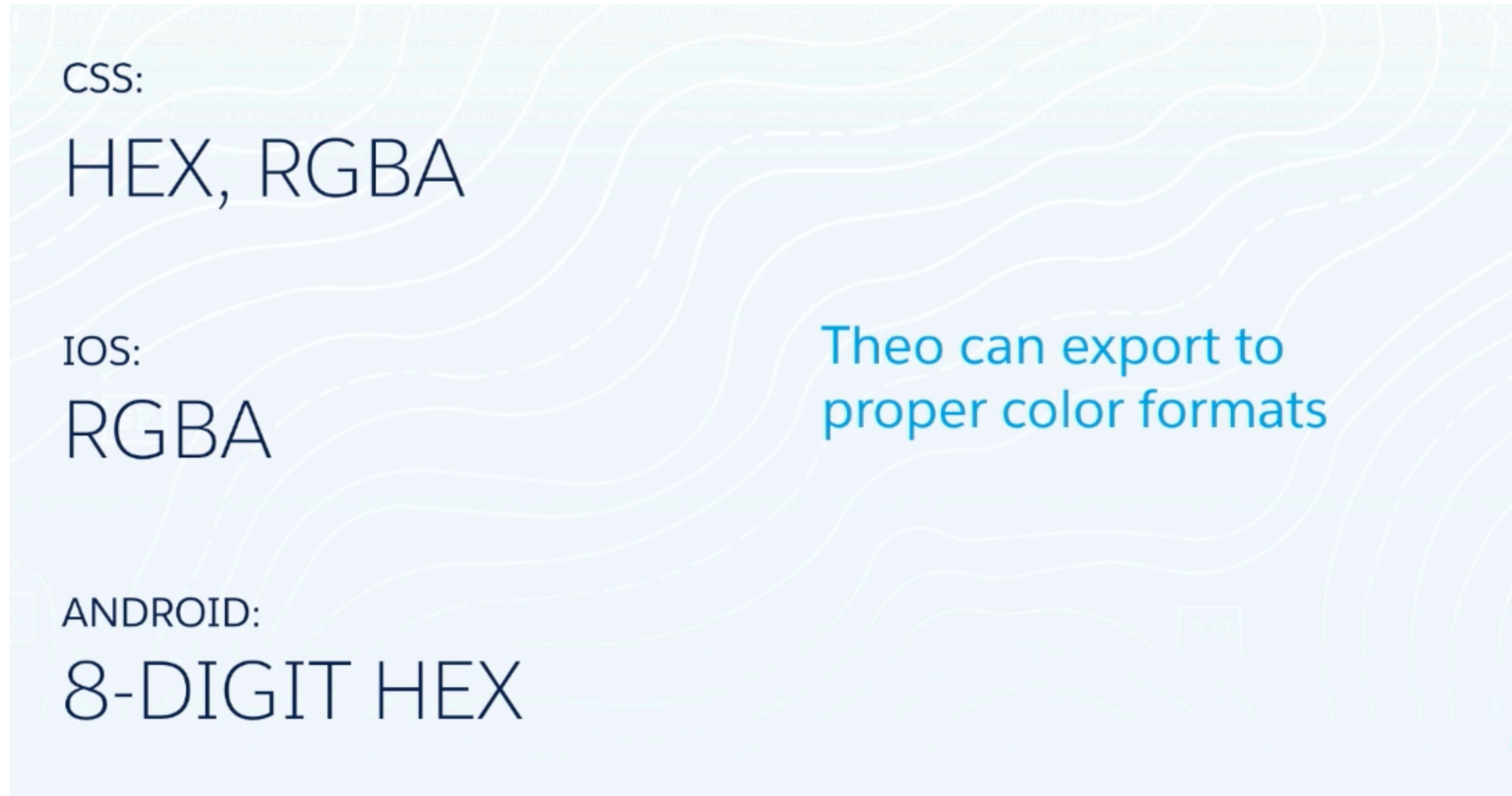
`$color-background-docked-panel-header`	rgb(255, 255, 255) #ffffff PALETTE_GRAY_1
`$global-header-color-background`	rgb(255, 255, 255) #ffffff PALETTE_GRAY_1

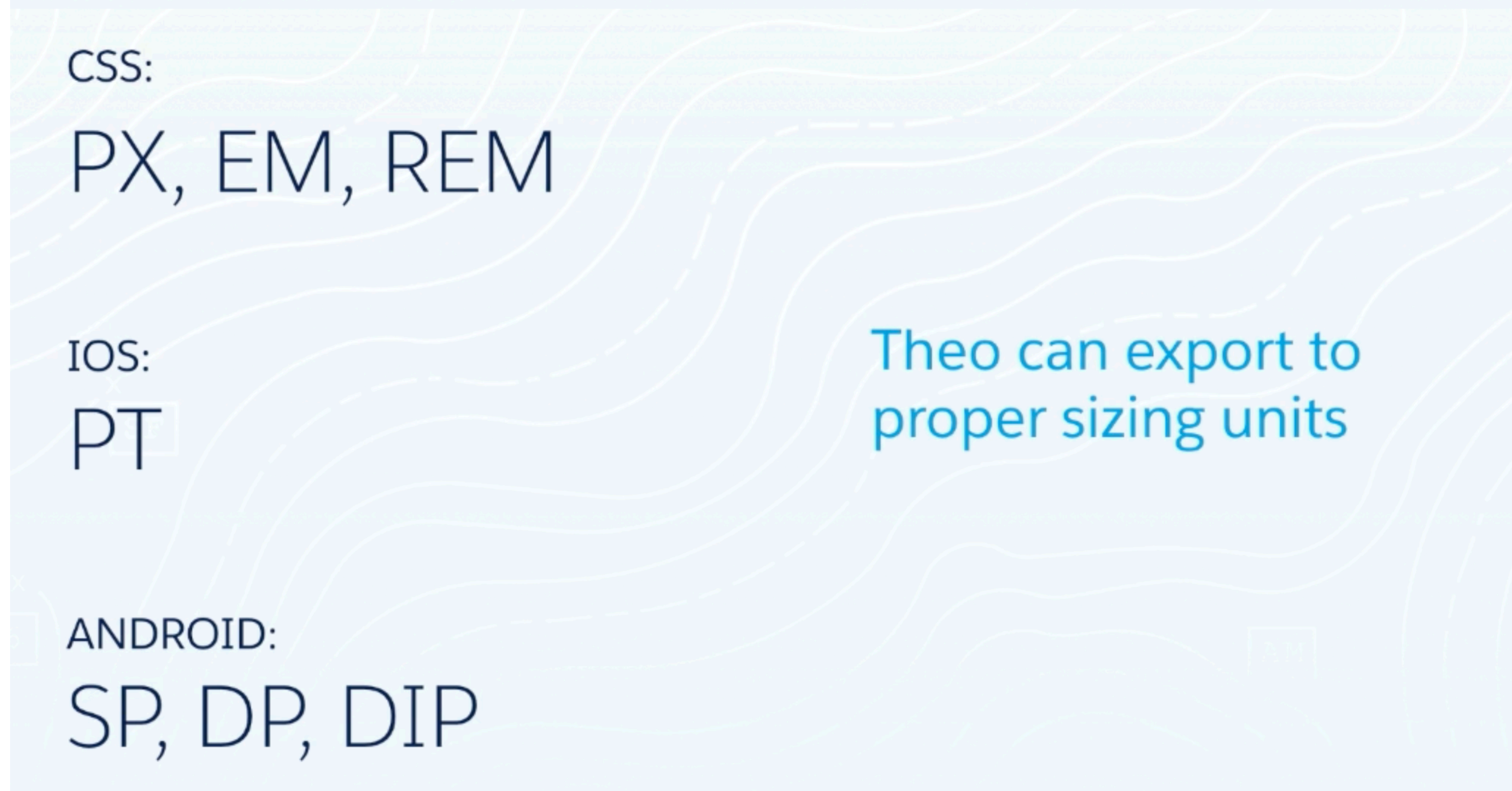
Automation

So, how do we make this system agnostic and scalable?

Salesforce does it through their custom generator called Theo. It generates design tokens for various technologies and by keeping the updating centralized from one place makes the users confident in altering them. The automation can work with the API to feed into final products and styleguide. If change gets made to the card, button pattern, that gets deemed to the styleguide and live site.



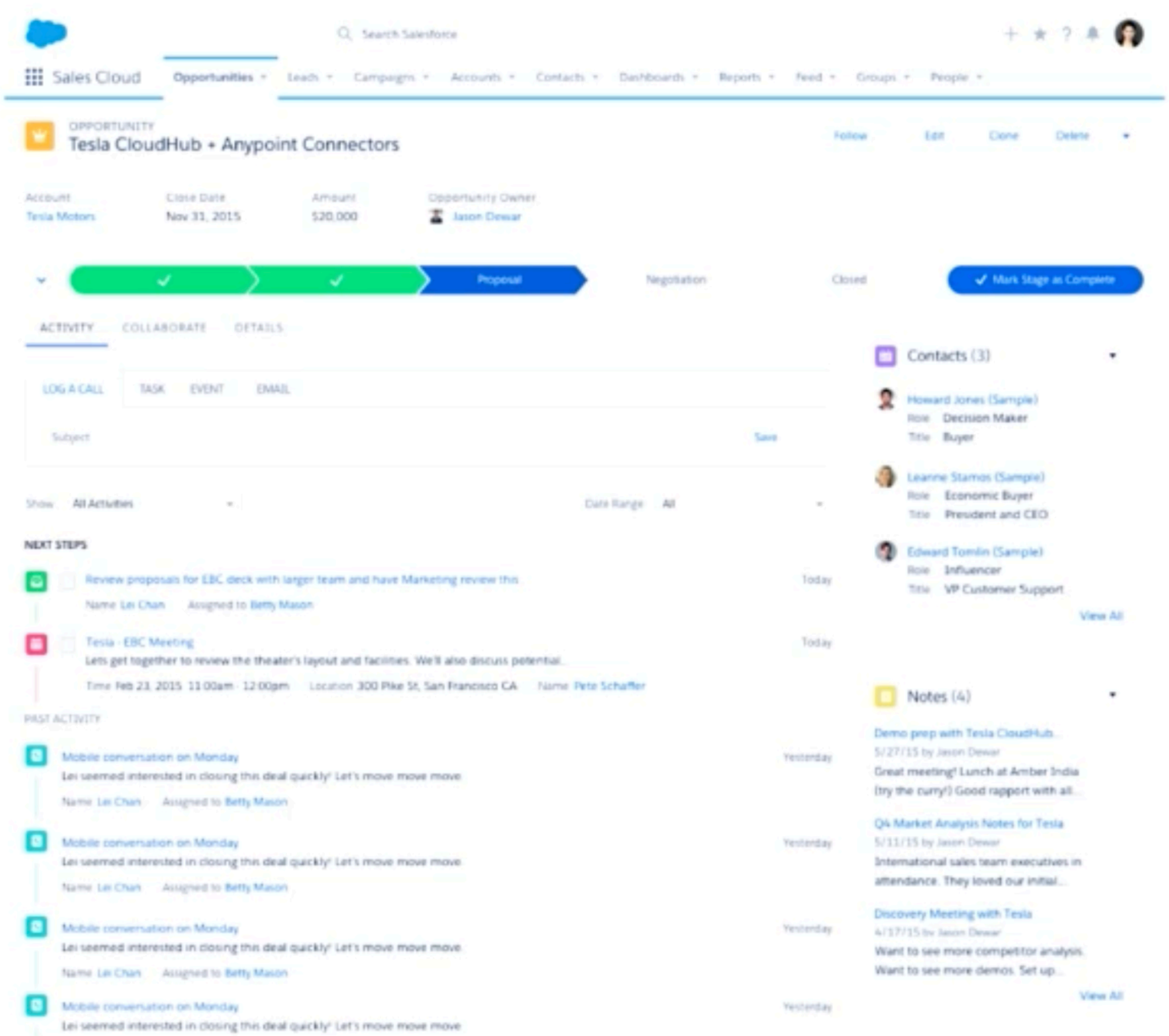




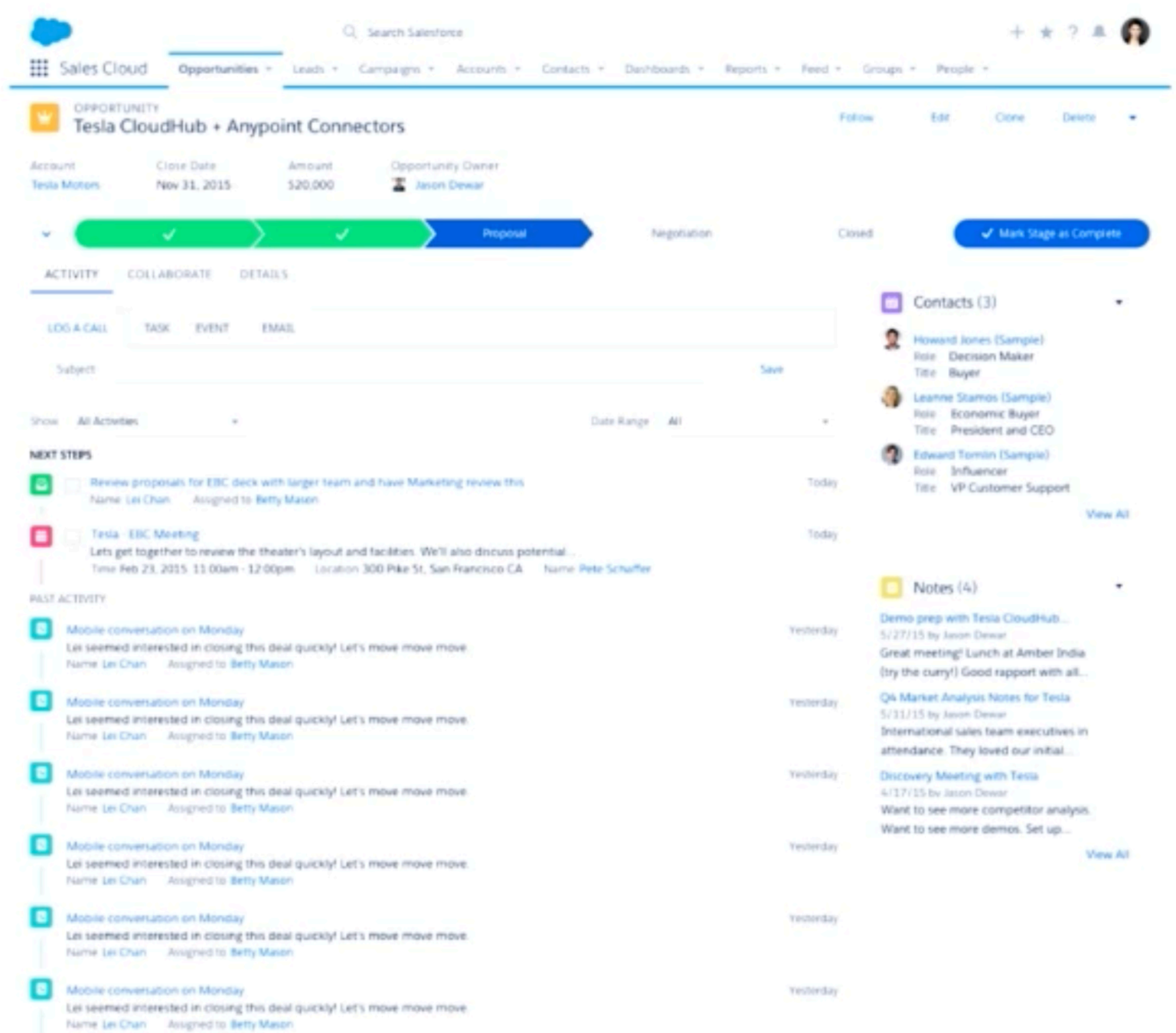
Change density

There are a lot of benefits that the tokens bring to the table in a design system. To demonstrate that Salesforce made their content display in a compact, more condensed view, or a more comfort layout, simply by changing a few spacing tokens. Very fast change that made an impact for many of their users who prefer this option found in gmail.

COMFORTABLE

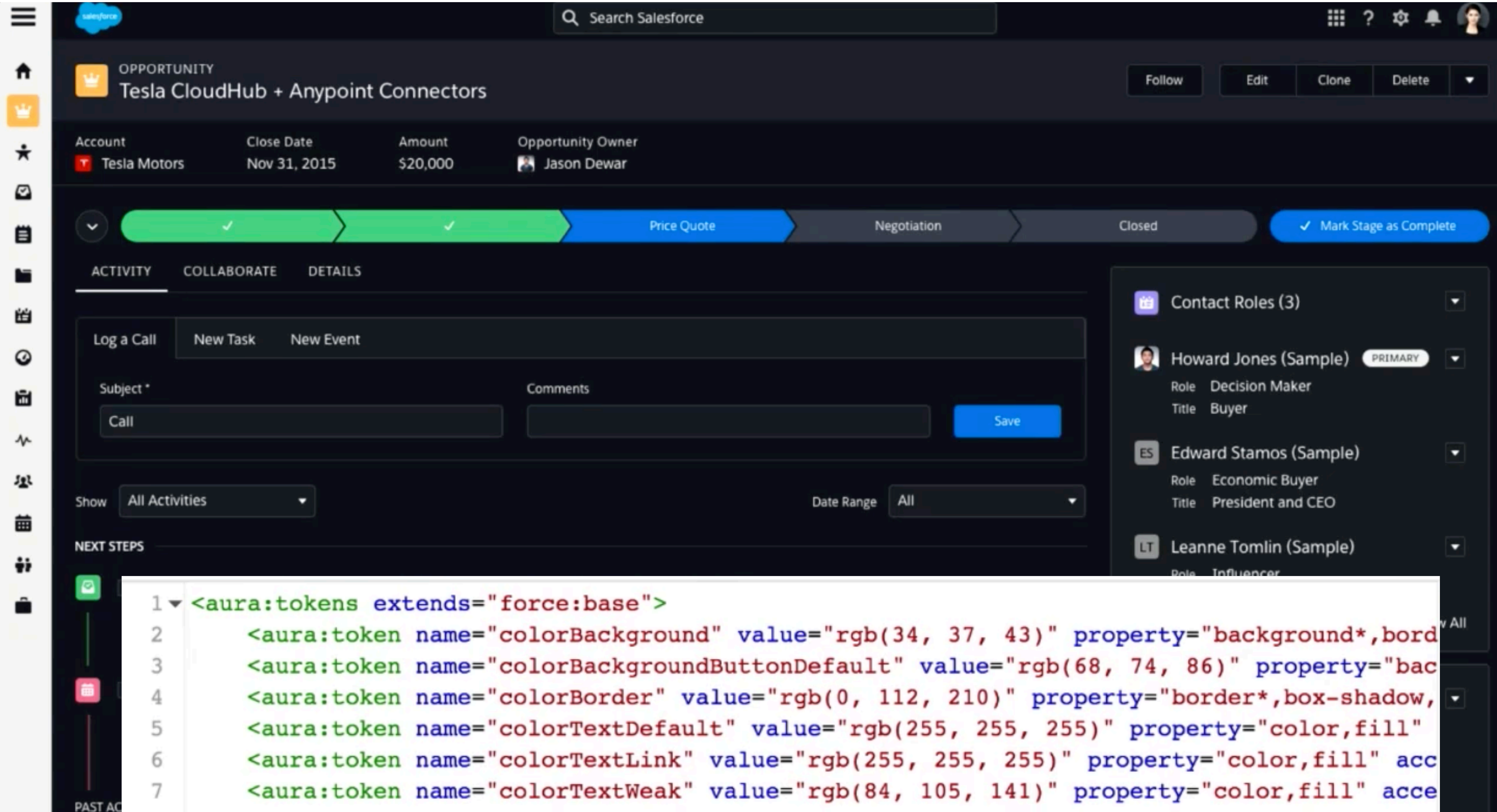


COMPACT



How about dark mode?

Another cool thing they showed in their demo i how they can turn their complete software to dark mode. Again, by simply overriding a few symbols they were able to achieve this result and put the design to the test.



Thanks for you attention!